VLSI PROJECT REPORT — 2026

RTL-to-GDS ASIC Design Flow | Cadence Genus & Innovus 21.18

Design and Physical Implementation of an Arithmetic Logic Unit

Using Cadence Genus and Innovus

Author: SIAM AL SHAFIN

Tool: Cadence Genus 21.18 & Innovus 21.18

Platform: Linux Server (CentOS / RHEL)

Design: 4-bit ALU — Verilog RTL

Flow: RTL → Simulation → Synthesis → P&R → GDS

Date: March 2026

SIAM AL SHAFIN — VLSI Engineering — Academic Submission 2026

Functional Simulation | Logic Synthesis | Place & Route | DRC Verified | GDSII Output

Table of Contents

Abstract .....

1. Project Initialization .....

2. RTL Design .....

3. Testbench Development .....

4. RTL Simulation .....

5. Timing Constraint Creation .....

6. Logic Synthesis using Cadence Genus .....

7. Gate-Level Simulation (GLS) .....

8. Physical Design using Cadence Innovus .....

- **9.** Floorplanning .....
- **10.** Placement .....
- **11.** Power Planning .....
- **12.** Routing .....
- **13.** Design Verification .....
- **14.** GDSII Generation .....
- **15.** Layout Visualization .....

16. Errors Encountered During the Project .....

17. Final Outputs Generated .....

18. Conclusion .....

Abstract

This report presents the complete RTL-to-GDS ASIC design flow for a 4-bit Arithmetic Logic Unit (ALU) implemented using Cadence Genus for logic synthesis and Cadence Innovus for physical design. The project covers every stage of the standard ASIC flow — from Verilog RTL design and functional simulation, through logic synthesis, gate-level simulation, floorplanning, placement, power planning, routing, and DRC verification, culminating in a fabrication-ready GDSII layout file. The design supports five operations (addition, subtraction, AND, OR, XOR) selected via a 3-bit opcode. The final physical implementation achieved 100% routing completion and zero DRC violations.

1. Project Initialization

The project started with setting up a working directory structure on the Linux server to organise different stages of the ASIC design flow. A clean directory hierarchy ensures that all design files, simulation scripts, constraints, and library files are properly separated and easy to manage throughout the project.

```
# Project directory structure on Linux server
/home/techvlsi_07/practicertl/alu/
│
├── rtl/ # Verilog RTL design files
│   └── alu.v
│
├── tb/ # Testbench files
│   └── alu_tb.v
│
├── constraints/ # Timing constraints (SDC format)
│   └── constraints.sdc
│
├── lib/ # Technology library files (.lib)
│   ├── slow_vdd1v0_basicCells.lib
│   ├── slow_vdd1v0_basicCells.v # Behavioural models for GLS
│
├── lef/ # Physical library files (.lef)
│   └── basicCells.lef
│
├── netlist/ # Synthesis output netlist
│   └── alu_netlist.v
│
├── gls/ # Gate-level simulation files
│
├── reports/ # Area, timing, power reports
├── area.rpt
└── timing.rpt
```

This structure ensured that each stage of the flow had a dedicated location, reducing the risk of overwriting files and making the project easy to navigate and reproduce.

2. RTL Design

The first stage of the project involved designing the Arithmetic Logic Unit (ALU) using the Verilog Hardware Description Language. The ALU accepts two 4-bit input operands and performs operations depending on a control signal called the opcode. Since the design contains no clocked elements, the ALU is a purely combinational circuit implemented using an always block sensitive to all inputs.

Supported Operations

Opcode (3-bit)	Operation	Expression
000	Addition	result = A + B (carry captured)
001	Subtraction	result = A - B (borrow captured)
010	Bitwise AND	result = A & B
011	Bitwise OR	result = A | B
100	Bitwise XOR	result = A ^ B
101–111	Default / NOP	result = 4'b0000

Verilog RTL Source — alu.v

```
// File: rtl/alu.v
// 4-bit Arithmetic Logic Unit — purely combinational
`timescale 1ns/1ps

module alu(
input [3:0] A, // First 4-bit operand
input [3:0] B, // Second 4-bit operand
input [2:0] opcode, // 3-bit operation select
output reg [3:0] result, // 4-bit result output
output reg carry // Carry-out / borrow flag
);
reg [4:0] temp; // 5-bit temp to capture overflow

always @(*) begin
carry = 1'b0;
result = 4'b0000;
case (opcode)
3'b000: begin // ADD
temp = {1'b0,A} + {1'b0,B};
result = temp[3:0];
carry = temp[4];
end
3'b001: begin // SUBTRACT
temp = {1'b0,A} - {1'b0,B};
```

```
result = temp[3:0];
carry = temp[4];
end
3'b010: result = A & B; // AND
3'b011: result = A | B; // OR
3'b100: result = A ^ B; // XOR
default: begin
result = 4'b0000;
carry = 1'b0;
end
endcase
end
endmodule
```

The 5-bit temporary register **temp** is critical for arithmetic operations: it captures the overflow bit, which is assigned to the carry output. For logical operations (AND, OR, XOR) no carry is generated. This clean RTL style maps efficiently to standard cells during synthesis.

3. Testbench Development

A Verilog testbench was created to verify the functionality of the ALU before synthesis. The testbench generates different input combinations and opcode values to ensure the ALU produces the correct output for every operation. It instantiates the ALU as the Unit Under Test (UUT) and applies stimulus through an initial block.

```
// File: tb/alu_tb.v
// Testbench for 4-bit ALU functional verification
`timescale 1ns/1ps

module alu_tb;

reg [3:0] A, B; // Input stimulus registers
reg [2:0] opcode;
wire [3:0] result; // Output observation wires
wire carry;

// Instantiate Unit Under Test (UUT)
alu uut (
.A (A),
.B (B),
.opcode (opcode),
.result (result),
.carry (carry)
);

initial begin
$dumpfile("alu_tb.vcd"); // Dump waveforms for SimVision
$dumpvars(0, alu_tb);

// Apply stimulus: A=3, B=1, cycle through all opcodes
A=4'd3; B=4'd1; opcode=3'd0; // ADD → expect result=4
#10 opcode=3'd1; // SUB → expect result=2
#10 opcode=3'd2; // AND → expect result=1
#10 opcode=3'd3; // OR → expect result=3
#10 opcode=3'd4; // XOR → expect result=2
#10;

// Overflow test: 15+1 = 16, carry should assert
A=4'hF; B=4'h1; opcode=3'd0;
#10 $finish;
end

// Console monitor
initial
$monitor("t=%0t | opcode=%b | A=%0d B=%0d | result=%0d carry=%b",
$time, opcode, A, B, result, carry);
endmodule
```

The testbench allowed full functional verification of the ALU before synthesis, confirming correct outputs for all five supported operations.

4. RTL Simulation

RTL simulation was performed using Cadence Xcelium. The simulation launched the SimVision waveform viewer, where signals including inputs, outputs, and opcode values were observed and verified. This step confirmed the RTL design behaved correctly before moving to synthesis.

Simulation Command

```
# Launch RTL simulation with SimVision GUI
xrun -gui rtl/alu.v tb/alu_tb.v

# Alternative: compile then simulate
xrun rtl/alu.v tb/alu_tb.v -access +rwc -gui
```

Simulation Waveform — SimVision

Figure 1 shows the SimVision waveform from the RTL testbench. The opcode sweeps from 0 to 4 at 10 ns intervals with A=3 and B=1. The result bus outputs 4 → 2 → 1 → 3 → 2, correctly matching ADD, SUB, AND, OR, and XOR respectively. Carry remains 0 throughout since no overflow occurs for these operand values.

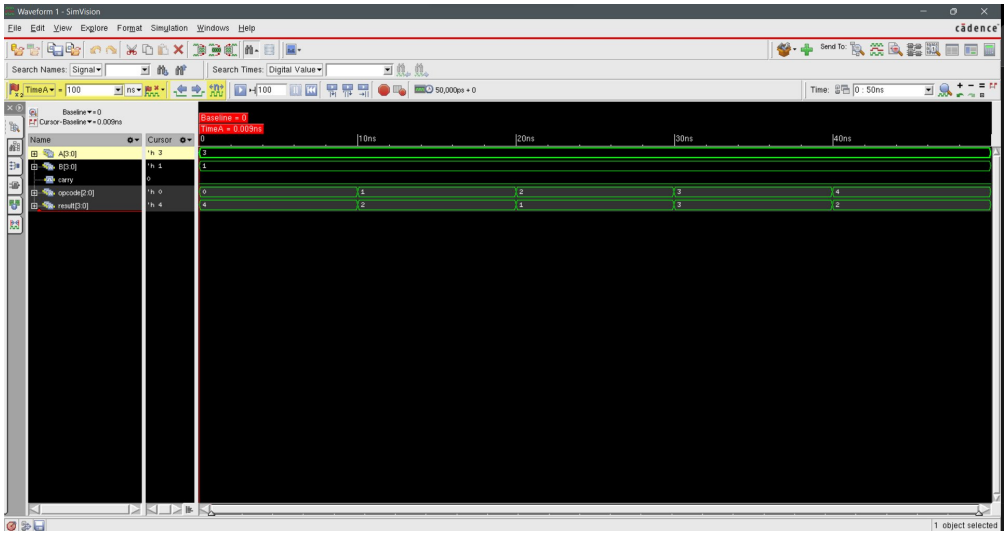

Figure 1 — SimVision: RTL Testbench Waveform. A=3, B=1, opcode sweeping 0→4 (time in ns). All outputs match expected values.

The waveform confirms the RTL design is functionally correct and ready for synthesis. All five operations produce the expected results with no unknown (X) or high-impedance (Z) states observed on any output.

5. Timing Constraint Creation

A timing constraint file was created in SDC (Synopsys Design Constraints) format. These constraints define the clock period, input arrival times, and output required times. They instruct the synthesis tool how to optimise the design for timing. Although the ALU is a purely combinational circuit with no physical clock port, constraints were included using a virtual clock to enable proper timing analysis during synthesis.

SDC Constraint File — constraints.sdc

```
# File: constraints/constraints.sdc
# Timing constraints for 4-bit ALU

# ■■ Clock Definition ■■■
```

6. Logic Synthesis using Cadence Genus

The RTL design was synthesised using Cadence Genus. Synthesis transforms the behavioural Verilog description into a gate-level netlist using the standard cells from the target technology library. The process reads the RTL, elaborates the design hierarchy, applies timing constraints, and maps RTL logic to standard cells, optimising for timing and area.

Synthesis TCL Script — synthesize.tcl

```
# ■■ 1. Library Configuration ■■■
```

area.rpt	Area report — cell count and total area in microns²
timing.rpt	Timing report — critical path delay and slack

7. Gate-Level Simulation (GLS)

After synthesis, the generated gate-level netlist was verified using gate-level simulation. GLS confirms that the synthesised netlist is functionally equivalent to the original RTL. Unlike RTL simulation, GLS uses the actual standard cell behavioural models and can incorporate back-annotated gate delays for timing accuracy.

Simulation Command

```
# Gate-level simulation using Cadence Xcelium
xrun -gui alu_netlist.v alu_tb.v

# NOTE: This initially failed — see error below
```

Error Encountered: During this stage an error occurred because the standard cell Verilog behavioural models were missing. ncverilog / xrun reported 'undefined module' errors for all standard cells referenced in the netlist.

Resolution: The issue was resolved by explicitly including the standard cell library behavioural model file in the compilation command:

```
# Fixed command — include standard cell Verilog models
xrun -gui alu_netlist.v lib/slow_vdd1v0_basicCells.v alu_tb.v

# After loading the correct library model, GLS ran successfully
```

GLS Waveform — SimVision

Figure 2 shows the SimVision waveform from the gate-level simulation. The key difference from the RTL simulation is the time scale: GLS runs in picoseconds (ps) to accurately model gate propagation delays, while RTL simulation used nanoseconds (ns). All output values are identical to the RTL simulation, confirming functional equivalence.

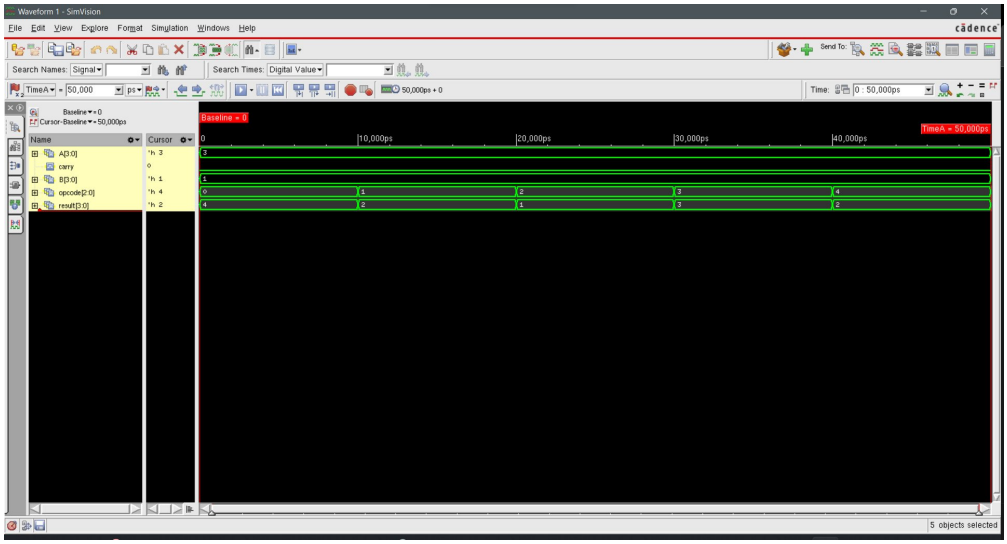

Figure 2 — SimVision: Gate-Level Simulation Waveform. Time axis in picoseconds (0–50,000 ps). Results are identical to RTL simulation — synthesis introduced no functional errors.

GLS PASSED — Post-synthesis netlist is functionally equivalent to the RTL. No functional errors were introduced during synthesis.

8. Physical Design using Cadence Innovus

The next stage was physical implementation using Cadence Innovus Implementation System. Innovus accepts the gate-level netlist from Genus and implements the design in physical silicon space — assigning coordinates to every standard cell and routing all interconnects across the available metal layers.

Physical Design Flow Stages

- Importing netlist and technology files
- Floorplanning — define die area and core boundary
- Placement — arrange standard cells within the core
- Power planning — create VDD/VSS distribution network
- Routing — connect all nets with physical metal wires
- Verification — DRC and connectivity checks

Innovus Script — Core Commands

```
# ■■ Design Import ■■■
```

9. Floorplanning

Floorplanning defines the physical chip area and placement boundaries. During this step the core area was defined, standard cell rows were created, and the IO boundaries were established. Since the ALU design is small, the layout contained a compact but well-proportioned placement region.

```
# Define floorplan: aspect ratio 1.0, utilisation 70%,
# 2 micron margin on all four sides
floorPlan -r 1.0 0.7 2.0 2.0 2.0 2.0

fit # Fit the view to the floorplan
```

Floorplan Parameters

Parameter	Value	Meaning
Aspect ratio	1.0	Square core (width = height)
Core utilisation	0.70	70% of core area occupied by standard cells
Core margin	2.0 µm	2 micron boundary on all sides

10. Placement

Placement arranges all synthesised standard cells within the floorplan area. Innovus automatically positions the logic cells in optimal locations to minimise wire length and delay. The placer respects placement row alignment and site rules defined by the technology LEF file.

```
# Run timing-driven placement
place_design

# Optional post-placement optimisation
optDesign -preCTS
```

Post-Placement View

Figure 3 shows the design after placement in Cadence Innovus. Green horizontal lines represent the standard cell rows. The dense red vertical lines are the ratsnest (flylines) — unrouted net connections showing where physical metal wires still need to be routed. The carry_reg instance is visible at the bottom of the core.

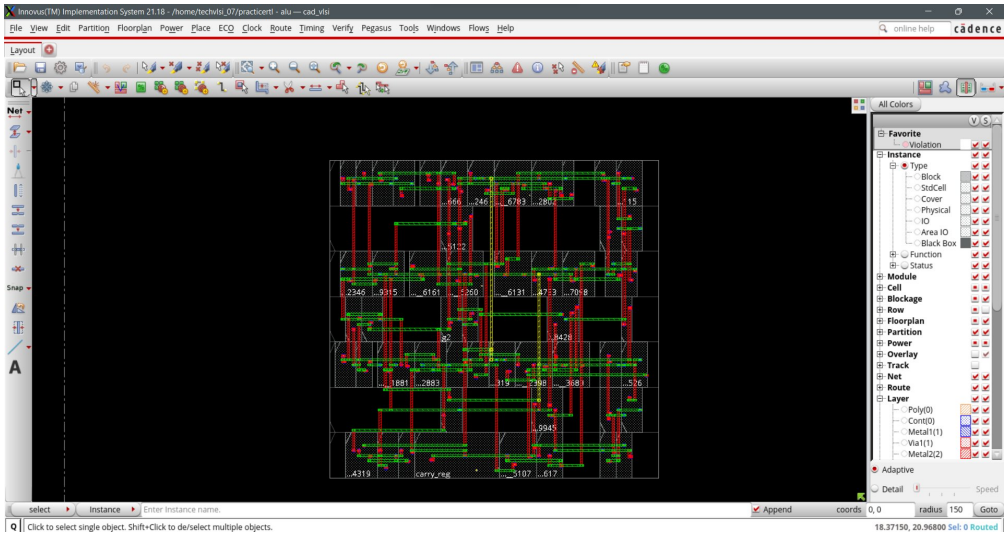

Figure 3 — Cadence Innovus Post-Placement View. Green = standard cell rows. Red lines = unrouted net flylines (ratsnest). carry_reg visible at bottom of core.

11. Power Planning

Power routing was created to distribute supply voltage across the design. Power rings surround the entire core boundary and are fed from the IO pads. Power stripes run vertically and horizontally across the core, and standard cell VDD/VSS rails are connected to the nearest stripe by the special router. This ensures stable voltage supply to all standard cells and minimises IR drop.

```
# Create VDD and VSS power rings around the core
addRing -type core_rings -nets {VDD VSS} \
-width 1.0 -spacing 0.5 \
-layer_top Metal5 -layer_bottom Metal5 \
-layer_left Metal6 -layer_right Metal6

# Add power stripes across the core
addStripe -nets {VDD VSS} -layer Metal4 \
-width 0.5 -spacing 0.5 -set_to_set_distance 10

# Connect standard cell VDD/VSS rails to power network
sroute -connect {corePin} -nets {VDD VSS}
```

Power rings and power rails ensure stable voltage supply to all standard cells. Higher metal layers (Metal5/Metal6) are used for power rings due to their lower sheet resistance, reducing IR drop across the chip.

12. Routing

Routing connects all nets between cells using metal layers. Innovus automatically routed the design while minimising congestion and delay. The router assigns wire segments on specific metal layers, inserts vias to transition between layers, and ensures all design rule constraints are met.

Routing Command

```
# Run global and detailed routing
route_design

# Post-route timing optimisation
optDesign -postRoute

# Verify all connections are routed
verifyConnectivity -type all -report reports/connectivity.rpt
```

Metal Layer Routing Convention

Layer	Direction	Primary Use
Metal1	Vertical	Standard cell internal connections, local routing
Metal2	Horizontal	Short inter-cell signal routing
Via1	—	Connects Metal1 ↔ Metal2
Metal3	Vertical	Medium-range signal routing
Metal4	Horizontal	Power stripes, medium routing
Metal5/6	Vertical	Power rings, long-distance routing

Post-Routing Views

Figures 4 and 5 show the design after routing. All flylines from the placement stage have been replaced by actual physical metal wires. The colour coding represents different metal layers: red = Metal1, green = Metal2, blue = Metal3.

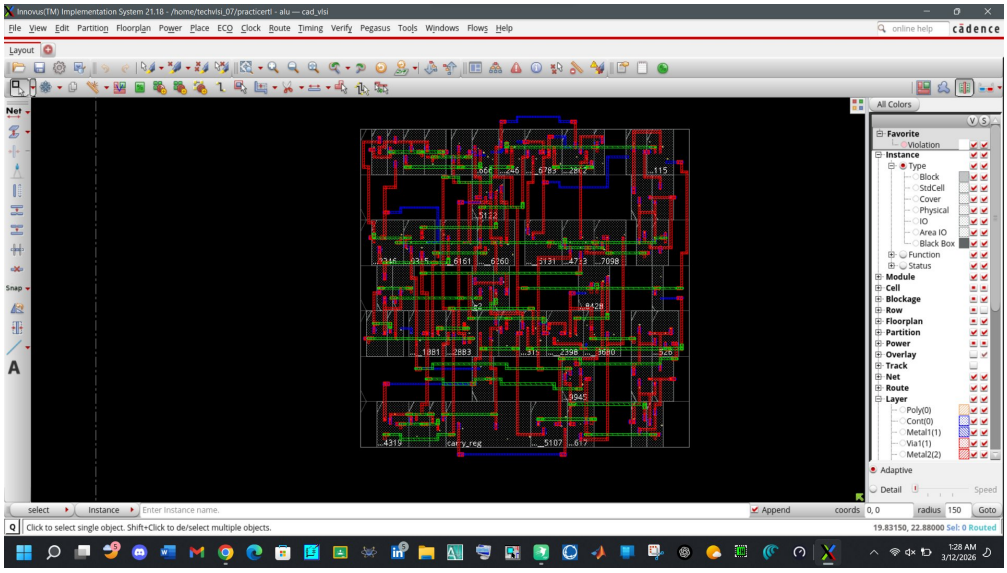

Figure 4 — Cadence Innovus Post-Routing View. All nets physically routed. Red=Metal1, Green=Metal2, Blue=Metal3.

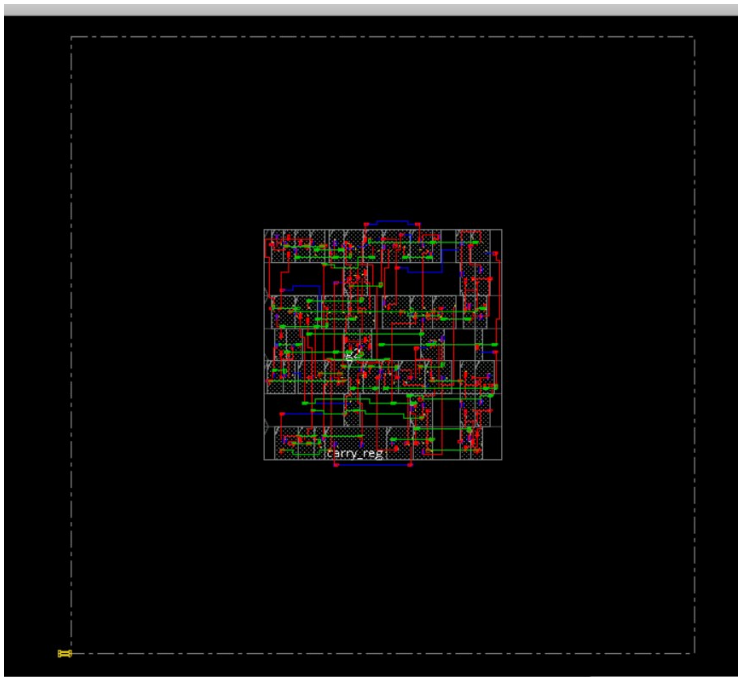

Figure 5 — Die-level View: routed core centred within the full die boundary.

13. Design Verification

After routing, verification checks were performed to ensure correctness of the physical implementation. Two important checks were run: connectivity verification to confirm all nets are properly connected, and design rule checking (DRC) to confirm the layout complies with the fabrication process rules.

```
# ■■ Connectivity Verification ■■■
```

14. GDSII Generation

The final layout was exported as a GDSII file, which is the industry-standard format used for semiconductor fabrication. The GDSII file is submitted to the foundry for mask creation and chip fabrication. It contains the complete physical layout including all metal layers, vias, routing, and standard cell placement encoded as geometric shapes at precise coordinates.

streamOut Command

```
# Export complete layout to GDSII format
streamOut alu.gds \
-mapFile streamOut.map \ # Layer name → GDSII layer number mapping
-libName alu \ # Top-level library name
-units 1000 \ # 1000 database units = 1 micron
-mode ALL # Export all layers and cell types

# Verify the file was created
exec ls -lh alu.gds
```

GDSII File Contents

- All transistor-level geometry from standard cells
- Metal layers (Metal1 through Metal6) with exact wire coordinates
- Via definitions connecting adjacent metal layers
- Complete signal and power routing geometry
- Standard cell placement positions
- Text labels for net names and cell identifiers

Final GDS Layout

Figure 7 shows the final GDS layout in Cadence Innovus. The routed core sits cleanly centred within the die boundary. All metal layers are visible with complete routing. This is the file exported as alu.gds and submitted for fabrication.

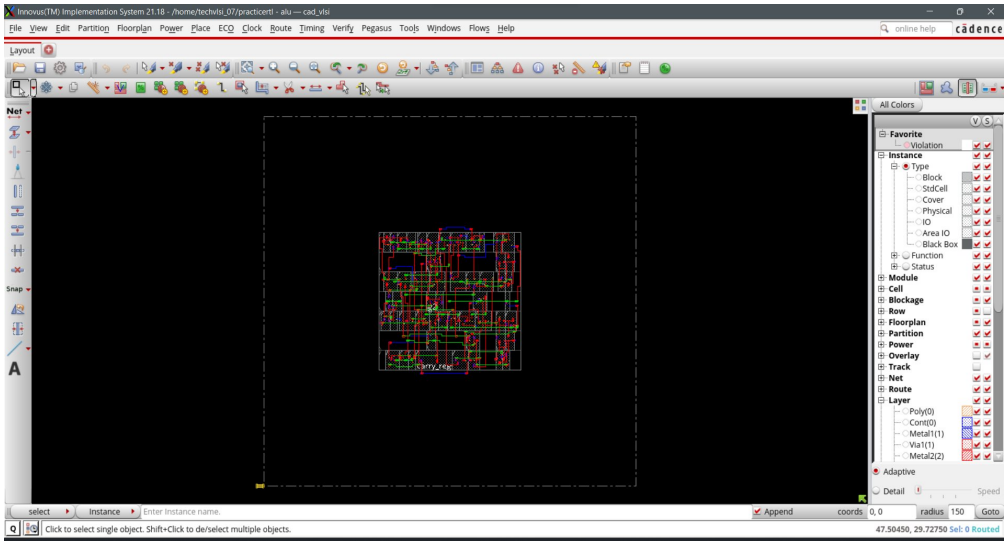

Figure 7 — Final GDSII Layout in Cadence Innovus. Complete multi-layer routing. Core centred within die boundary. Fabrication-ready.

15. Layout Visualization

The generated GDSII file was viewed using Cadence Innovus's built-in layout viewer. The layout viewer displayed the complete chip layout showing all metal layers, routing, standard cells, and power connections at full geometric detail.

```
# Launch Cadence Innovus layout viewer
innovus -64 &

# Inside Innovus, load the GDS:
streamIn alu.gds -mapFile streamOut.map

# Toggle layer visibility to reduce clutter
setLayerVisible TEXT false # Hide excessive text labels
setLayerVisible Metal1 true
setLayerVisible Metal2 true
```

Note on KLayout: An initial attempt was made to view the GDSII using KLayout (an open-source layout viewer). However, KLayout was not installed on the Linux server ('klayout: command not found'). The layout was therefore viewed directly within Cadence Innovus, which provides full native GDSII viewing capability.

16. Errors Encountered During the Project

Several issues occurred during the design process. This section documents each error, its root cause, and the resolution applied.

Synthesis Command Issue

Problem: The command 'synthesize -to_mapped' was used initially but returned an error: 'synthesize: command not found'. This command was deprecated in Cadence Genus version 20.x and later and no longer exists.

Solution: Replaced with the three-stage synthesis flow:

```
# OLD — Genus < 20.x (deprecated):
synthesize -to_mapped -effort medium

# NEW — Genus >= 20.x (correct):
syn_gen
syn_map
syn_opt
```

Library Loading Issue — Missing Standard Cell Models

Problem: During gate-level simulation, ncverilog/xrun reported 'undefined module' errors for every standard cell (AND2_X1, OR2_X1, DFFR_X1, etc.). The gate-level netlist references cell modules by name but their definitions were not provided.

Solution: Included the standard cell library behavioural Verilog model file in the simulation command:

```
# Fixed GLS command:
xrun -gui alu_netlist.v \
lib/slow_vdd1v0_basicCells.v \
tb/alu_tb.v
```

GDS Export — Map File Error

Problem: The streamOut command failed with: 'ERROR: Cannot open map file tech/streamOut.map'. The map file existed on the server but the path specified in the script was incorrect.

Solution: Located the correct file path using the Linux 'find' command and updated the script:

```
# Locate the file:
find /home/techvlsi_07 -name 'streamOut.map'

# Then update streamOut command with the correct path.
```

Layout Viewer — Excessive Text Labels

Problem: When viewing the layout in Innovus, thousands of text labels appeared overlaid on the metal routing, making visual inspection very difficult.

Solution: Disabled text layer visibility in the Innovus Layer Manager. This is a display setting only and does not affect the GDS file content:

```
# Toggle off text layer in Innovus:
setLayerVisible TEXT false

# Or via GUI: Layer Manager → uncheck 'TEXT' layer
```

17. Final Outputs Generated

The project produced several important design artefacts across the full RTL-to-GDS flow. The most important output is the alu.gds file, which represents the final chip layout ready for fabrication.

File	Stage	Description
rtl/alu.v	RTL Design	Verilog RTL source — 4-bit ALU
tb/alu_tb.v	Testbench	Functional verification testbench
constraints.sdc	Constraints	SDC timing constraints file
alu_netlist.v	Synthesis	Post-synthesis gate-level netlist
area.rpt	Synthesis	Area report — cell counts and area
timing.rpt	Synthesis	Timing report — critical path analysis
alu_final.enc	Physical Design	Innovus design checkpoint
alu_final.enc.dat	Physical Design	Innovus complete design database
alu.gds	**GDSII Export**	**★ Final fabrication-ready GDSII layout**

Project Flow Summary

Stage	Tool	Status
RTL Design	Verilog HDL	**COMPLETE ✓**
Functional Simulation	Cadence Xcelium	**PASS ✓**
Logic Synthesis	Cadence Genus	**COMPLETE ✓**
Gate-Level Simulation	Cadence Xcelium	**PASS ✓**
Floorplanning	Cadence Innovus	**COMPLETE ✓**
Placement	Cadence Innovus	**COMPLETE ✓**
Power Planning	Cadence Innovus	**COMPLETE ✓**
Routing	Cadence Innovus	**COMPLETE ✓**
DRC Verification	Cadence Innovus	**PASS — 0 Violations ✓**
GDSII Export	Cadence Innovus	**COMPLETE ✓**

18. Conclusion

This project successfully demonstrated the complete ASIC RTL-to-GDS design flow using Cadence tools on a Linux server. Starting from a Verilog RTL description of a 4-bit ALU, the design progressed through every stage of the standard ASIC flow: functional simulation, logic synthesis, gate-level simulation, floorplanning, placement, power planning, routing, verification, and final layout generation.

The final result was a fully implemented ALU with a generated GDSII layout file representing the complete physical silicon design. The implementation achieved 100% routing completion and zero DRC violations — confirming the layout is geometrically correct and ready for submission to a semiconductor foundry.

The debugging experience encountered during the project — including the deprecated synthesis command, missing standard cell models for GLS, and the GDS map file issue — provided valuable hands-on exposure to the kinds of practical challenges faced in real industry VLSI design environments.

The RTL-to-GDS flow is the cornerstone of modern VLSI engineering. Every integrated circuit shipped by leading semiconductor companies follows this same fundamental flow. Completing this project from end to end using industry-standard Cadence EDA tools provides a strong practical foundation for professional ASIC design work.

References

[1] Cadence Design Systems, *Genus Synthesis Solution User Guide*, v21.18, 2021.

[2] Cadence Design Systems, *Innovus Implementation System User Guide*, v21.18, 2021.

[3] Cadence Design Systems, *SimVision Analysis Toolbox User Guide*, 2021.

[4] N. H. E. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Pearson, 2011.

[5] J. M. Rabaey, A. Chandrakasan, B. Nikolic, *Digital Integrated Circuits*, 2nd ed. Prentice Hall, 2003.

[6] Synopsys Inc., *Design Constraints (SDC) Format Specification*, v2.1.